Source: docs\DELIVERY_UPDATE.md | Export: 11 November 2026

Administrator Delivery Update

Local implementation of the approved Stitch drawer/panel, preserving the 192px sidebar, approximately 58px header, local Lora font and compact admin controls. No additional scale reduction, dependency, environment variable, function endpoint, index or security rule is required.

Entry and interaction

Both Order Details **Update Delivery** buttons open the panel. Dispatch/Mark delivered header shortcuts also open it. The general order-action form remains available for existing workflows. Opening performs a fresh authorized `adminOrders:get`; each order has its own panel state. The real reference, state, recipient and saved delivery fields are shown.

Desktop uses a 480px right drawer; mobile uses a full-width panel with a scrollable form and separate safe-area footer. A single native modal owns focus throughout edit, review and discard steps. The background is locked; closing restores focus without scrolling. Escape returns from review to edit, or asks about discarding actual edits. Switching tabs/windows does not ask questions or reset fields. Actual page unload with unsaved/uncertain work gets the browser's normal protection. In-app navigation is stopped for discard review; after closing, repeat the navigation action.

Update Status requires explicitly selecting one of the engine's allowed progressive transitions. Cancellation/rejection and returns remain in the existing workflow. **Edit Details Only** submits `command: delivery-details` and explicitly states that status is unchanged. Neither mode invents Packed, Ready to Dispatch or Out for Delivery states.

Review lists the reference, status change or unchanged status, courier, AWB, safe external link, ETA and explicit clearing. Back to Edit preserves inputs. Confirm Update is the only write. No-op edits are rejected. Success appears only after a server response.

Schema and server eligibility

Existing top-level fields remain authoritative: `courier`, `trackingNumber`, `trackingUrl`. The sole schema addition is optional `estimatedDeliveryDate: string | null`, using `YYYY-MM-DD` date-only semantics. Missing legacy values render **Not set**; no backfill is needed. The client and server share validation in `src/orders/delivery.ts`.

- Metadata edits are allowed only in Placed, Confirmed, Processing and Shipped.
- Delivered, Cancelled and Rejected are terminal/read-only in this panel. Returns keep their existing operations.
- Courier and AWB are trimmed and limited to 100 characters. AWB accepts letters, numbers, spaces and `. _ / # ( ) + -` separators.
- Tracking links must be credential-free HTTPS, at most 500 characters. No URL is fetched by the server. External links use `noopener noreferrer`.
- Dispatch validates the complete resulting courier/AWB record, including saved values. A shipped order cannot lose its required courier/AWB through a metadata edit. The drawer also requires these values when marking a shipped order Delivered.
- ETA is optional. Omitted fields preserve values; `null` explicitly clears ETA. Empty strings clear optional courier/tracking fields where state requirements permit, and clear tracking URLs.
- Newly changed ETA values must be real calendar dates from today through 730 days ahead, using **Asia/Kolkata** for today's date. An unchanged overdue estimate remains valid. Display reads date components directly, without converting midnight through a timezone.

The existing `adminOrders:update` transaction verifies administrator permission, revision and the exact-request operation receipt. Both the `admin: true` claim and enabled `adminAccess/{uid}` record are still required. Updates and history are atomic, with server-derived actor/timestamps and before/after delivery changes in administrator audit notes. Identical retries do not duplicate history; changed payloads cannot reuse a receipt.

Delivery-only updates preserve status, COD state, collection, financial snapshots, items and inventory. Delivered alone still does not collect COD. No automatic courier integration, SMS, email, Razorpay operation or refund was added.

Refresh, conflict and customer visibility

Successful writes use the existing admin change signal to refresh Order Details, list/dashboard and other tabs. Background refresh never overwrites draft values. A changed revision blocks a new write; Reload current order and Review latest revision preserve the draft and require a new review. Unknown/lost responses lock edits and offer **Retry exact update safely** using the same operation ID and payload.

Customer tracking retains verified UID ownership. It shows the saved courier/AWB, safe link and ETA marked as an estimate. Administrator actor IDs, internal notes and delivery audit notes remain excluded from customer history projections.

Design adaptations and local acceptance

The screenshots and exported HTML guide the hierarchy, segmented modes, fields, review and footer. Production references can be longer than the prototype's PS-1024, so they wrap safely. The existing ivory/maroon Lora baseline takes precedence over prototype lavender/font/CDN settings. Simulated status bars, prototype state controls and unsupported SMS promises were omitted.

Run `npm.cmd run dev:api` and `npm.cmd run dev` with the existing local setup. Use an isolated emulator administrator/order, open `/admin/orders/{id}`, and check both Update Delivery entry points. Test review/back/cancel, dispatch with saved data, detail-only edits, ETA add/change/clear, a second-tab conflict, exact retry and authorized customer tracking. Never use a production order as a mutation fixture.

Automated coverage is integrated in `scripts/admin-emulator-suite.ts`: `delivery-update-emulator.test.ts` and `delivery-update-browser.test.py`, alongside existing regressions. Screenshots are written under ignored `.cache/delivery-update-browser/` at 320, 390, 768, 1024, 1440 and 1920px, at normal browser zoom. Unit coverage is in `src/orders/model.test.ts`.

Validation completed

- Typecheck (`npm.cmd run lint`), production build and all 88 unit tests passed.
- Eight focused emulator suites passed (nine tests total): administrator/category/customer rules, orders/COD, Order Details, Orders List, dashboard and Delivery Update. The delivery suite additionally passed in the final focused run. These cover dual administrator authorization, dispatch requirements, legacy dates, ETA persistence/clearing, terminal restrictions, atomic revision conflicts, exact replay, private-field projection, customer ownership and unchanged COD/inventory/financial state.
- Six affected browser suites passed across focused runs: Delivery Update, Order Details, Orders List, dashboard, COD and purchase journey. Orders List had one earlier confirmation timeout; its isolated final rerun passed. The complete all-feature `test:admin` suite was not run as one uninterrupted invocation.
- Delivery screenshots at all six widths were captured; representative desktop, mobile, 320px and short 320×560px views were visually inspected. No horizontal overflow or footer obstruction was found. Checks include focus containment/restoration, review/edit/discard, no dialog on tab/window switching, dirty background refresh, exact lost-response retry, failed-read retry, saved customer ETA/link display and sign-out cleanup.
- The pinned Vercel Node builder packaged the actual API entry point and invoked it locally without a tsx loader. Runtime initialization/status handling passed its enabled/disabled ESM regression probes. This is local packaging evidence, not a hosted deployment test.
- `git diff --check` passed. No rules/indexes or production state were read or changed for this implementation.

Changed files:

- `src/admin/orders/DeliveryPanel.tsx`, `delivery.css`, `OrderDetails.tsx`, `OrderActions.tsx`.
- `src/orders/delivery.ts`, `model.ts`, `model.test.ts`, `ui.tsx`.
- `server/orders.ts`.
- `scripts/delivery-update-emulator.test.ts`, `delivery-update-browser.test.py`, `delivery-update-fixture-control.ts`, `order-details-browser.test.py`, `admin-emulator-suite.ts`.
- `docs/DELIVERY_UPDATE.md`, `ORDER_DETAILS.md`.

No migration, rule/index publication or new external setup is needed. Hosted acceptance requires the owner to deploy the local application changes separately. This task does not commit, push, deploy, publish or modify production records/users.